

User application guideline

Step	Action	Description	Expected Outcome	Page/Redirect
1	Access the Registration Page	Navigate to the registration page (usually /register) to create a new user account.	Displays the registration form.	/register
2	Fill in Registration Form	Enter required details (such as username, email, password) into the form.	Form data is collected and validated.	Form fields are populated.
3	Submit Registration Form	Submit the form by clicking the Submit or Register button.	The form is validated. If valid, a new user is created.	Redirects to the "ping_operation" page after successful registration.
4	Successful Registration	After form submission, the system registers the new user and logs them in automatically.	The user is logged in, starting a session.	Redirected to the "ping_operation" page.
5	Access the Login Page	Navigate to the login page (usually /login) if the user already has an account.	Displays the login form.	/login
6	Fill in Login Credentials	Enter username and password into the login form.	Form data is entered for login authentication.	Form fields are populated.
7	Submit Login Form	Click the Login button after entering credentials.	The system attempts to authenticate the user.	Redirects based on successful authentication or failure.

8	Authentication Success	If the credentials are correct, the user is authenticated and logged in.	A session is created for the authenticated user.	Redirected to the page specified in the "next" URL parameter (defaults to "ping_operation" if no "next" is specified).
9	Authentication Failure	If the credentials are incorrect, the user is not authenticated.	The system will display an error message or re-render the login form.	Stays on the login page with error feedback.
10	Accessing Protected Pages	After successful login, access other parts of the application (e.g., "ping_operation" page) as an authenticated user.	User has access to protected resources.	"ping_operation" or other secured pages.

Key Notes

- **User Registration:** When a new user is registered, they are automatically logged in and redirected to the dashboard ([ping_operation](#) page).
- **Login Process:** If the user already exists, they can log in using their credentials. The login page may redirect to the original page the user tried to access before being prompted to log in (through the [next](#) URL).
- **Error Handling:** In case of incorrect login credentials, the user remains on the login page with an error message.

Ping Operation Application Guideline

Operation	Description	Result
Enable Ping	A ping test is sent to check if the target device or host is reachable over the network.	- Device is alive: The device responds to the ping, indicating it is reachable. - Device is unreachable: The device does not respond, indicating it is either offline or there is a network issue.

Verbose Ping Result	A more detailed ping output, often used to diagnose problems by showing round-trip time and other related stats.	- Success : The ping is successful with information like round-trip time. - Failure : The ping fails with error messages indicating the nature of the failure (e.g., timeout, unreachable destination).
Traceroute Result	A traceroute command is run to identify the path taken by packets to reach the target destination.	- Success : The traceroute provides each hop along the path, showing latency at each point. - Failure : Traceroute fails to reach the destination, often due to routing issues or blocked ports along the path.
Forward DNS Lookup Result	A DNS lookup is performed to resolve a domain name to its corresponding IP address.	- Success : The DNS server returns the IP address corresponding to the domain name. - Failure : The DNS server cannot resolve the domain name, possibly due to an invalid domain or DNS issues.
Reverse DNS Lookup Result	A reverse DNS lookup is performed to resolve an IP address to a domain name.	- Success : The IP address is successfully mapped to a domain name. - Failure : No domain name is found for the IP, possibly due to lack of reverse DNS record for the IP address.
Verbose Forward DNS Lookup Result	A more detailed version of a DNS lookup that provides information from the DNS server directly.	- Success : Detailed DNS information, such as TTL (time to live), authoritative servers, and IP addresses for the domain. - Failure : DNS server returns an error message with failure details (e.g., NXDOMAIN, timeout).
Verbose Reverse DNS Lookup Result	A more detailed reverse DNS lookup result that provides insights about the DNS server and resolution process.	- Success : The result contains detailed information on the reverse lookup, including TTL, authoritative nameservers, and the associated domain name. - Failure : Details on failure, such as no records or timeout errors from DNS servers.
SNMP Walk Result	The result of an SNMP walk, querying multiple devices or systems to gather data about their status and configurations.	- Success : SNMP queries return information about device configurations, status, and metrics. - Failure : No response or error messages, which can happen due to incorrect SNMP community strings, network access restrictions, or device misconfiguration.

Simple SNMP Get Result	A simple SNMP operation where a query is made to retrieve a specific piece of information (e.g., device uptime, CPU usage).	- Success : The requested information (e.g., device uptime or CPU utilization) is returned from the device. - Failure : No response or error messages, which can occur if the SNMP agent is misconfigured, the device is unreachable, or the SNMP community string is incorrect.
-------------------------------	---	--

Simple SNMP Operation:

- **SNMP Get**: This is typically a one-time query to retrieve a specific value from a network device. Commonly used to retrieve things like system uptime, device version, or other operational statistics. It requires the device to be configured with SNMP support, and the correct community string must be used.

Example of Simple SNMP Get:

Command: A simple SNMP get query might request the system uptime:

```
snmpget -v 2c -c public <device-ip> 1.3.6.1.2.1.1.3.0
```

-
- **Output:**
 - **Success**: Returns the uptime value of the device in the response.
 - **Failure**: Error message, such as "Timeout" or "No such object," if the device is unreachable or doesn't respond as expected.

Explanation of Results:

- **Success**: If SNMP is enabled and the community string is correct, the result will include the requested value, such as system uptime (in seconds), CPU load, or any other SNMP-supported data.
- **Failure**: If there's a misconfiguration (wrong community string, device not reachable, or SNMP disabled on the device), the operation will return an error, such as a timeout or unreachable host error.

Common Errors in SNMP:

- **Timeout**: The SNMP request couldn't reach the device, possibly due to firewall settings or device being offline.
- **No such object**: The requested SNMP OID (Object Identifier) doesn't exist on the device or the device isn't configured to provide that data.
- **Access Denied**: The community string used for authentication is incorrect or doesn't have the necessary permissions.

DNS application guideline

Below is a detailed table format guideline for the DNS management application based on the provided Django views:

Feature	Description	Implementation Details	Expected Outcome
DNS List View	Displays a list of saved DNS records on the <code>dns.html</code> page.	- Query the database to get DNS records using <code>DNS.objects.values("dns_name", "dns_created_at")</code> .	- The page will show all DNS names and their creation date if any DNS records exist. - If no records exist, it will show an empty page.
Add DNS	Allows users to add a new DNS record.	- Retrieve the DNS address from <code>request.POST.get("dns_address")</code> . - Create a new DNS record using <code>DNS.objects.create(dns_name=get_dns_id)</code> .	- A new DNS record is created with the provided DNS address and added to the database. - The page redirects to the DNS list view.
Delete DNS	Allows users to delete a DNS record based on its unique ID.	- Fetch the DNS object using <code>get_object_or_404(DNS, dns_name=dns_id)</code> . - Delete the object using <code>dns.delete()</code> .	- The DNS record corresponding to the provided DNS name is deleted from the database. - The page redirects to the DNS list view.

Error Handling for DNS Deletion	Provides error handling if a DNS record does not exist when attempting to delete it.	- Use <code>get_object_or_404()</code> to handle the case where the DNS record may not exist, and prevent a <code>DoesNotExist</code> error.	- If the DNS record exists, it is deleted. - If not, a 404 error is triggered.
Logging	Logs DNS-related actions for debugging and tracking purposes.	- Utilize Python's <code>logging</code> library to log the DNS operations (<code>add_dns</code> and <code>delete_dns</code>).	- Log entries will be created for each DNS addition or deletion attempt.

DNS View Implementation:

- **URL Path:** `/dns/`
- **Function:** Displays the list of DNS records saved in the database.
- **Template:** `dns.html`
 - The DNS records are displayed by looping through `get_saved_dns` (which contains `dns_name` and `dns_created_at`).
- **Context Data:** The list of DNS names and their creation date is passed as context.
- **Outcome:** Renders the DNS list or an empty page if no records are found.

Add DNS Implementation:

- **URL Path:** `/dns/add/`
- **Function:** Adds a new DNS record based on the `dns_address` provided in the POST request.
- **Input:** A form where users can input a DNS address (`dns_address`).
- **Action:**
 - The `dns_address` value is retrieved from `request.POST.get("dns_address")`.
 - A new DNS record is created using `DNS.objects.create(dns_name=get_dns_id)`.
- **Outcome:** Redirects to the DNS list view after the DNS record is successfully added.

Delete DNS Implementation:

- **URL Path:** `/dns/delete/<dns_id>/`

- **Function:** Deletes a DNS record identified by `dns_id` passed via URL.
- **Action:**
 - Fetch the DNS object by `dns_name` using `get_object_or_404(DNS, dns_name=dns_id)`.
 - Deletes the object using `dns.delete()`.
- **Outcome:** Redirects to the DNS list view after deletion. If the DNS record is not found, a 404 error will be raised.

Error Handling:

- **Deleting Non-Existent DNS:** The `get_object_or_404()` function ensures that the application handles cases where the DNS record does not exist. This prevents server errors and displays a 404 page instead.

Logging:

Log Entries: The logging mechanism will track DNS creation and deletion operations. For example:

```
logger.info(f"DNS {get_dns_id} added.")
logger.info(f"DNS {dns_id} deleted.")
```

- These logs will be helpful for debugging and keeping track of DNS actions.